Storm Intensity Distribution — Test Results

Automated Test Documentation for Model Governance

MKM Research Labs

Version 1.0 — 23-March-2026

David K Kelly

Legal Notice

PROPRIETARY AND CONFIDENTIAL

1 Test Results — Storm Intensity Distribution (MKM-SI-001)

Tests run on **23 March 2026 at 11:32:07**.

Table 1: Test Results Summary — Storm Intensity Distribution (MKM-SI-001)

Total Tests	105
Passed	105
Failed	0
Skipped	0
Duration	0.24s

Table 2: Detailed Test Results — Storm Intensity Distribution

Test Description	Acceptance Criteria	Result	Time
Custom parameters	`d.params.base_mean == 50`; `d.params.tail_threshold == 70`	Pass	0.000s
Default parameters	`d.params.base_mean == 45.0`; `d.params.base_std == 15.0` (+4 more)	Pass	0.000s
From params object	`d.params.base_mean == 55`	Pass	0.000s
From scenario baseline	`d is not None`; `d.params.name == "baseline"`	Pass	0.000s
From scenario unknown raises	—	Pass	0.000s
Seed gives deterministic output	`s1 == s2`	Pass	0.000s
Tail prob between 0 and 1	`0.0 < d._tail_prob < 1.0`	Pass	0.000s
All categories defined	`set(DURATION_PARAMS.keys()) == expected`	Pass	0.000s
Catastrophic extended	`high == 240`	Pass	0.000s
Each tier's max roughly doubles the previous.	`1.5 <= ratio <= 3.0`	Pass	0.000s
Extreme extended	`DURATION_PARAMS["extreme"] == (36, 72, 144)`	Pass	0.000s
Moderate extended	`low == 12`; `mode == 30` (+1 more)	Pass	0.000s
Severe extended	`DURATION_PARAMS["severe"] == (24, 48, 96)`	Pass	0.000s
Gap types	`GapType.SHORT.value == "short"`; `GapType.MEDIUM.value == "medium"` (+1 more)	Pass	0.000s
Sequence gap mapping	`SEQUENCE_GAP_TYPE[SequenceType.DOUBLET] == GapType.MEDIUM`; `SEQUENCE_GAP_TYPE[SequenceType.CLUSTER] == GapType.MEDIUM` (+2 more)	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Sequence types	`SequenceType.ISOLATED.value == "isolated"; SequenceType.DOUBLET.value == "doublet"` (+2 more)	Pass	0.000s
Storm count ranges	`SEQUENCE_TYPE_STORM_COUNTS[SequenceType.ISOLATED] == (1, 1); SEQUENCE_TYPE_STORM_COUNTS[SequenceType.DOUBLET] == (2, 2)` (+2 more)	Pass	0.000s
Above max is zero	`d.exceedance_probability(d.params.max_intensity + 10) == 0.0`	Pass	0.000s
Above threshold uses pareto	`0 < p < 1`	Pass	0.000s
At max intensity is zero	`d.exceedance_probability(d.params.max_intensity) == 0.0`	Pass	0.000s
At min intensity is one	`d.exceedance_probability(d.params.min_intensity) == 1.0`	Pass	0.000s
At threshold continuous	`abs(p_just_below - p_just_above) < 0.05`	Pass	0.000s
Below min is one	`d.exceedance_probability(d.params.min_intensity - 5) == 1.0`	Pass	0.000s
Below threshold uses normal	`0 < p < 1`	Pass	0.000s
Monotonically decreasing	`all(probs[i] > probs[i + 1] for i in range(len(probs) - 1))`	Pass	0.000s
Constants	`EVENT_WINDOW_HOURS == 168; MIN_DRAINAGE_WINDOW_HOURS == 12` (+1 more)	Pass	0.000s
Exact boundary	`fits_in_window([78.0, 78.0], [0.0])`	Pass	0.000s
Exceeds window	`not fits_in_window([72.0, 72.0], [48.0])`	Pass	0.000s
Simple doublet fits	`fits_in_window([24.0, 24.0], [48.0])`	Pass	0.000s
Single storm	`fits_in_window([100.0], [])`	Pass	0.000s
Long gaps	`GAP_PARAMS[GapType.LONG] == (48, 96, 144)`	Pass	0.000s
Medium gaps	`GAP_PARAMS[GapType.MEDIUM] == (24, 48, 72)`	Pass	0.000s
Short gaps	`GAP_PARAMS[GapType.SHORT] == (6, 18, 36)`	Pass	0.000s
Doublet medium	`get_gap_range(SequenceType.DOUBLET) == (24, 48, 72)`	Pass	0.000s
Isolated zero	`get_gap_range(SequenceType.ISOLATED) == (0, 0, 0)`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Get duration range	`get_duration_range("moderate") == (12, 30, 60)`	Pass	0.000s
Get max duration	`get_max_duration("catastrophic") == 240; get_max_duration("minimal") == 12`	Pass	0.000s
Ids unique	`len(ids) == 100`	Pass	0.000s
Sequence id format	`sid.startswith("STORM-"); len(sid) == 14`	Pass	0.000s
Sequence ids unique	`len(ids) == 100`	Pass	0.000s
Storm id format	`sid.startswith("STORM-"); len(sid) == 14`	Pass	0.000s
One prob returns min	`d.inverse_exceedance(1) == d.params.min_intensity`	Pass	0.000s
Result in bounds	`d.params.min_intensity <= v <= d.params.max_intensity; d.params.min_intensity <= v <= d.params.max_intensity`	Pass	0.000s
Roundtrip above threshold	`abs(recovered - target) < 3.0`	Pass	0.000s
Roundtrip below threshold	`abs(recovered - target) < 2.0`	Pass	0.000s
Zero prob returns max	`d.inverse_exceedance(0) == d.params.max_intensity`	Pass	0.000s
Fat tail higher exceedance	`p_fat > p_thin`	Pass	0.000s
Pareto above threshold between 0 and 1	`0 < p < 1`	Pass	0.000s
Pareto at threshold is one	`d._pareto_exceedance(d.params.tail_threshold) == 1.0`	Pass	0.000s
Pareto below threshold is one	`d._pareto_exceedance(d.params.tail_threshold - 5) == 1.0`	Pass	0.000s
100yr greater than 10yr	`i100 > i10`	Pass	0.000s
Ordering 10 50 100	`i10 < i50 < i100`	Pass	0.000s
Result in bounds	`d.params.min_intensity <= v <= d.params.max_intensity; d.params.min_intensity <= v <= d.params.max_intensity`	Pass	0.000s
Storms per year affects result	`many > few`	Pass	0.000s
Invalid category	—	Pass	0.000s
Mode should be near the most frequent value in a large sample.	`abs(median - mode) < 15`	Pass	0.005s
Line 50: rng=None → creates np.random.RandomState() internally.	`low <= d <= high`	Pass	0.000s
Reproducibility	`d1 == d2`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
All sampled durations fall within [min, max] for the category. [category=’baseline’]	`low <= d <= high`	Pass	0.000s
All sampled durations fall within [min, max] for the category. [category=’catastrophic’]	`low <= d <= high`	Pass	0.000s
All sampled durations fall within [min, max] for the category. [category=’extreme’]	`low <= d <= high`	Pass	0.000s
All sampled durations fall within [min, max] for the category. [category=’minimal’]	`low <= d <= high`	Pass	0.000s
All sampled durations fall within [min, max] for the category. [category=’moderate’]	`low <= d <= high`	Pass	0.000s
All sampled durations fall within [min, max] for the category. [category=’severe’]	`low <= d <= high`	Pass	0.000s
Doublet gaps should fall in medium range (24-72h).	`all(24 <= g <= 72 for g in gaps)`	Pass	0.000s
Isolated returns zero	`sample_gap(SequenceType.ISOLATED) == 0.0`	Pass	0.000s
Line 49: rng=None → creates np.random.RandomState() internally.	`low <= g <= high`	Pass	0.000s
Persistent gaps should fall in short range (6-36h).	`all(6 <= g <= 36 for g in gaps)`	Pass	0.000s
Reproducibility	`g1 == g2`	Pass	0.000s
Within bounds[sequencetype.cluster] [seq_type=¡SequenceType.CLUSTER: ’cluster’¿]	`low <= g <= high`	Pass	0.000s
Within bounds[sequencetype.doublet] [seq_type=¡SequenceType.DOUBLET: ’doublet’¿]	`low <= g <= high`	Pass	0.000s
Within bounds[sequencetype.persistent] [seq_type=¡SequenceType.PERSISTENT: ’persistent’¿]	`low <= g <= high`	Pass	0.000s
Different seeds different output	`d1.sample(10) != d2.sample(10)`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Sample n returns correct length	`len(samples) == 100`	Pass	0.000s
Sample single	`d.params.min_intensity <= v <= d.params.max_intensity`	Pass	0.000s
Samples within bounds	`all(d.params.min_intensity <= s <= d.params.max_intensity...`	Pass	0.001s
Seeded reproducibility	`s1 == s2`	Pass	0.000s
Create basic	`storm.storm_id == "STORM-abc12345"; storm.duration_hours == 24.0`	Pass	0.000s
From dict round trip	`restored.storm_id == original.storm_id; restored.duration_hours == original.duration_hours` (+6 more)	Pass	0.000s
Precipitation rounding	`d["precipitation_mm"] == 35.12; d["intensity_factor"] == 1.234568` (+1 more)	Pass	0.000s
To dict	`d["storm_id"] == "STORM-test1234"; d["start_time_hours"] == 48.0` (+6 more)	Pass	0.000s
Describe returns string	`isinstance(desc, str); "base_mean" in desc.lower() or "Base mean" in desc`	Pass	0.073s
Params roundtrip	`recovered.base_mean == 52; recovered.name == "custom"`	Pass	0.000s
Tail probability matches internal	`d.to_dict()['tail_probability'] == pytest.approx(d._tail_...`	Pass	0.000s
To dict keys	`'parameters' in result; 'tail_probability' in result`	Pass	0.000s
Antecedent defaults	`seq.antecedent_soil_moisture == "normal"; seq.antecedent_groundwater == "normal"`	Pass	0.000s
Create doublet	`seq.num_storms == 2; len(seq.storms) == 2` (+2 more)	Pass	0.000s
From dict round trip	`restored.storm_id == original.storm_id; restored.duration_hours == original.duration_hours` (+6 more)	Pass	0.000s
Isolated no gaps	`len(seq.inter_storm_gaps_hours) == 0`	Pass	0.000s
Spatial correlation defaults	`seq.spatial_correlation_enabled is False; seq.spatial_correlation_range_km is None`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Spatial correlation enabled	`d["spatial_correlation_enabled"] is True; d["spatial_correlation_range_km"] == 43.2`	Pass	0.000s
To dict	`d["storm_id"] == "STORM-test1234"; d["start_time_hours"] == 48.0` (+6 more)	Pass	0.000s
Exceedance probs decreasing	`stats['prob_exceed_60'] >= stats['prob_exceed_70'] >= ...`	Pass	0.000s
Fat tail higher p99 than thin	`fat['p99'] > thin['p99']`	Pass	0.146s
Mean in reasonable range	`30 < stats['mean'] < 70`	Pass	0.000s
Percentiles ordered	`stats['p10'] <= stats['p25'] <= stats['p50'] <= \{} ...`	Pass	0.000s
Required keys present	`k in stats`	Pass	0.000s
Duration mismatch	`any("total_duration_hours" in e for e in errors)`	Pass	0.000s
Exceeds precip window	`any("MAX_PRECIP_END_HOUR" in e for e in errors)`	Pass	0.000s
Invalid intensity category	`any("invalid intensity" in e for e in errors)`	Pass	0.000s
Invalid sequence type	`any("Invalid sequence_type" in e for e in errors)`	Pass	0.000s
Overlapping storms	`any("starts at" in e and "before storm" in e for e in err...`	Pass	0.000s
Storm count mismatch	`any("num_storms" in e for e in errors)`	Pass	0.000s
Too many storms	`any("not in [1, 5]" in e for e in errors)`	Pass	0.000s
Valid doublet	`errors == []`	Pass	0.000s

1.1 Test Document History

Date	Version	Description	Author
05-Mar-2026	1.0	Initial test suite (12 tests)	D. Kelly
07-Mar-2026	1.1	73 test(s) added; 12 test(s) removed (total: 146)	D. Kelly
08-Mar-2026	1.2	42 test(s) added (total: 115)	D. Kelly
12-Mar-2026	1.3	2 test(s) added (total: 117)	D. Kelly
23-Mar-2026	1.4	12 test(s) removed (total: 105)	D. Kelly